Proiect HDL
Etapa 0

top.v

```
`timescale 1ns / 1ns

module top(
    input           MAX10_CLK1_50,
    input [1:0]     KEY,

    output [7:0]    HEX0,
    output [7:0]    HEX1,
    output [7:0]    HEX2,
    output [7:0]    HEX3,
    output [7:0]    HEX4,
    output [7:0]    HEX5
);

    localparam          TP          = 1;
    localparam          IN_FREQ     = 50_000_000;  // 50 MHz


    // Clock divider to create a 1Hz signal
    clock_divider #(
        .FREQ_IN    (IN_FREQ        ),
        .FREQ_OUT   (1              ),
        .MAKE_PULSE (1              )
    ) clock_divider_0 (
        .clk        (MAX10_CLK1_50  ),
        .reset_i    (~KEY[0]        ),
        .clk_o      (slow_clk       )
    );


    hex_handler hex_handler_inst(
        .clk        (MAX10_CLK1_50  ),
        .reset      (~KEY[0]        ),
        .slow_clk   (slow_clk       ),

        .HEX0       (HEX0           ),
        .HEX1       (HEX1           ),
        .HEX2       (HEX2           ),
        .HEX3       (HEX3           ),
        .HEX4       (HEX4           ),
        .HEX5       (HEX5           )
    );

endmodule
```

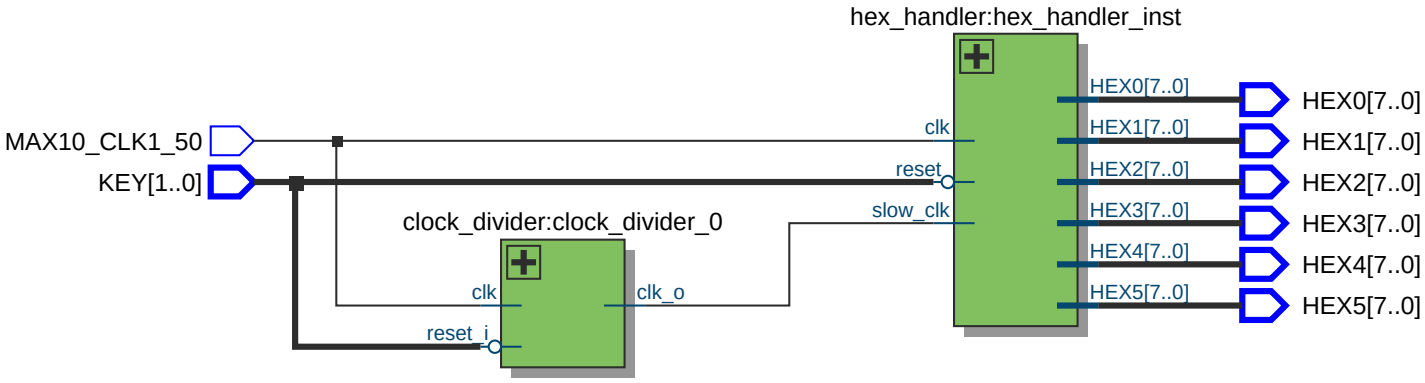
clock_divider.v

```
`timescale 1ns / 1ns

module clock_divider #(
    parameter FREQ_IN    = 100_000_000,  // 100 MHz
    parameter FREQ_OUT   = 9600,
    parameter MAKE_PULSE = 0
)(
    input       clk,
    input       reset_i,
    output reg  clk_o
);

    localparam COUNT_MAX = FREQ_IN / FREQ_OUT;
    reg [$clog2(COUNT_MAX)-1:0] counter;


    // Clock divider counter
    always @(posedge clk or posedge reset_i) begin
        if (reset_i)
            counter  <= {$clog2(COUNT_MAX){1'b0}};
        else if (counter == COUNT_MAX - 1)
            counter  <= {$clog2(COUNT_MAX){1'b0}};
        else
            counter  <= counter + 1;
    end

    // Output clock signal
    always @(posedge clk) begin
        if(MAKE_PULSE)
            clk_o <=  (counter == COUNT_MAX/2);
        else
            clk_o <=  (counter < COUNT_MAX/2);
    end

endmodule
```

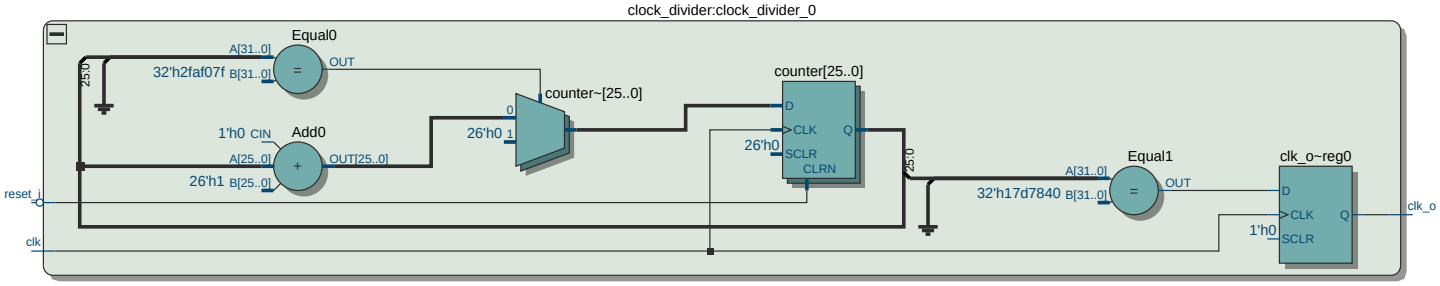
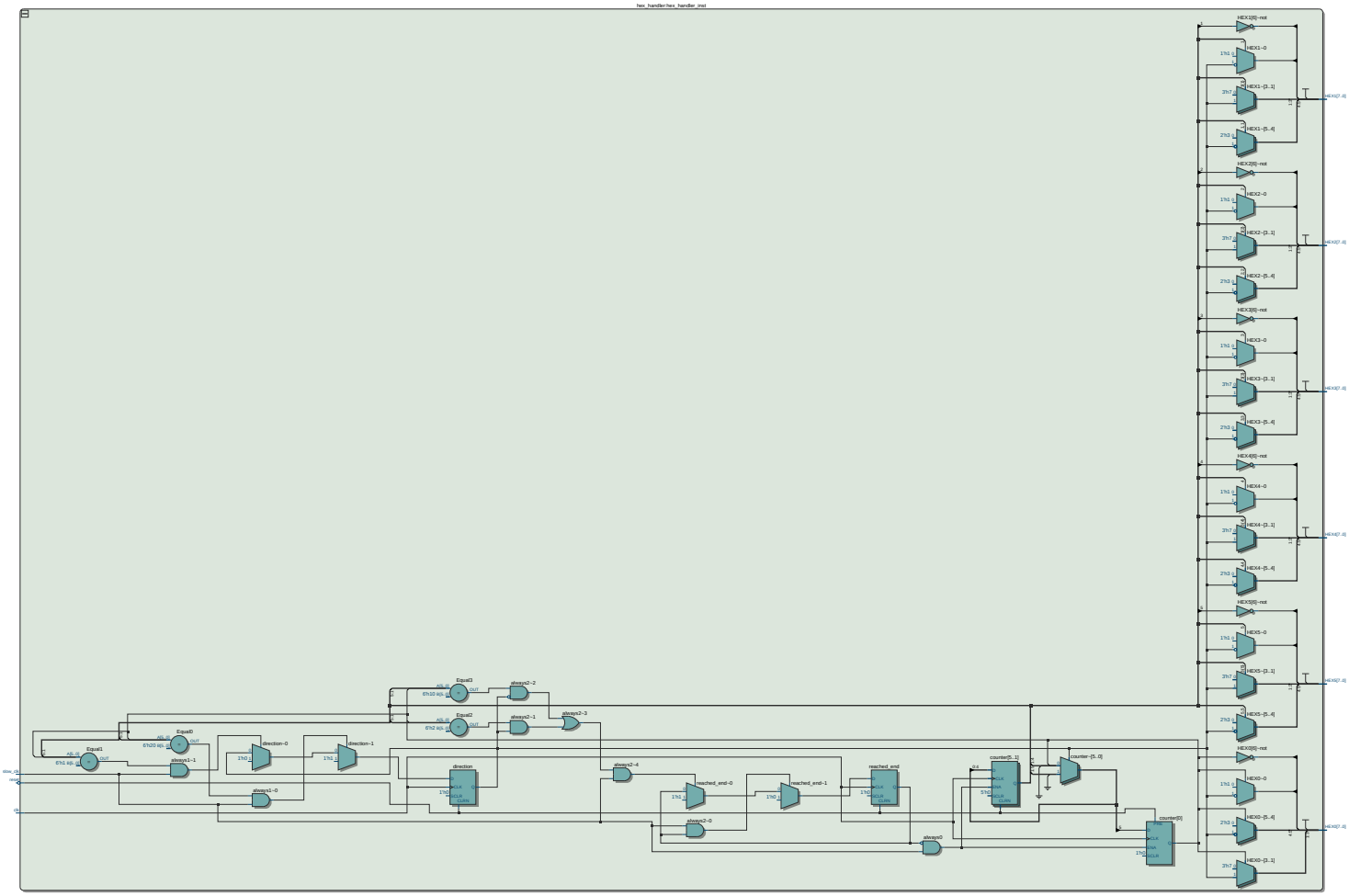
hex_handler.v

```
`timescale 1ns / 1ns

module hex_handler(
    input           clk,
    input           reset,
    input           slow_clk,

    output [7:0]    HEX0,
    output [7:0]    HEX1,
    output [7:0]    HEX2,
    output [7:0]    HEX3,
    output [7:0]    HEX4,
    output [7:0]    HEX5
);

    localparam           TP          = 1;
    localparam [8-1 : 0] HEX_BALL_LO = 8'b10100011;
    localparam [8-1 : 0] HEX_BALL_HI = 8'b10011100;
```

```
    reg [6-1 : 0]   counter;
    reg             direction;
    reg             reached_end;


    // 6-bit shift register
    always @(posedge clk or posedge reset) begin
        if(reset)
            counter <= 6'b1;
        else if(slow_clk & (~reached_end))
            counter <= direction ? (counter >> 1) : (counter << 1);
    end

    // direction register
    always @(posedge clk or posedge reset) begin
        if(reset)
            direction <= 0;
        else if(slow_clk & (counter == 6'b100000))
            direction <= 1;
        else if(slow_clk & (counter == 6'b000001))
            direction <= 0;
    end

    // reached_end register - to stall the counter one clock cycle
    always @(posedge clk or posedge reset) begin
        if(reset)
            reached_end <= 0;
        else if(slow_clk & reached_end)
            reached_end <= 0;
        else if(slow_clk & ((direction & counter == 6'b000010) |
(~direction & counter == 6'b010000)))
            reached_end <= 1;
    end


    // 7-segment displays pattern generator
    assign HEX0 = (counter[0]) ? (direction ? HEX_BALL_HI : HEX_BALL_LO) :
{8{1'b1}};
    assign HEX1 = (counter[1]) ? (direction ? HEX_BALL_HI : HEX_BALL_LO) :
{8{1'b1}};
    assign HEX2 = (counter[2]) ? (direction ? HEX_BALL_HI : HEX_BALL_LO) :
{8{1'b1}};
    assign HEX3 = (counter[3]) ? (direction ? HEX_BALL_HI : HEX_BALL_LO) :
{8{1'b1}};
    assign HEX4 = (counter[4]) ? (direction ? HEX_BALL_HI : HEX_BALL_LO) :
{8{1'b1}};
    assign HEX5 = (counter[5]) ? (direction ? HEX_BALL_HI : HEX_BALL_LO) :
{8{1'b1}};

endmodule
```


hex_handler:hex_handler_inst
MAX10_CLK1_50
KEY[1..0]
clk
reset
slow_clk
HEX0[7..0]
HEX1[7..0]
HEX2[7..0]
HEX3[7..0]
HEX4[7..0]
HEX5[7..0]
clock_divider:clock_divider_0
clk
reset_i
clk_o

clock_divider:clock_divider_0



hex_handler:hex_handler_inst